

GAT ANDRES BONIFACIO HIGH SCHOOL



How the Project Works

A practical onboarding guide for new Nexora groupmates

Live-reconciled edition · July 18, 2026 · capstone-nest-react-lms

Contents

How the Project Works	4
1. Executive summary	4
The one rule to remember	4
2. How the pieces fit together	5
The request path	5
What each technology contributes	5
Trust boundaries	6
3. The repository at a glance	6
Start at these files	6
4. What happens when a user signs in	6
Web session	6
Mobile session	7
Why refresh rotation exists	7
5. Understanding the core features	8
5.1 Classes, modules, and lessons	8
5.2 Class Record	8
5.3 Grading formulas	9
5.4 Assessments	9
5.5 Performance recompute	9
5.6 LXP and interventions	10
5.7 AI Mentor, quizzes, and extraction	10
5.8 BullMQ asynchronous jobs	10
6. Day-to-day developer guide: where do I edit?	12
I want to add or change a web page	12
I want to add or change a mobile screen	12
I want to add an API endpoint	12
I want to create a database table	13
I want to modify an AI prompt	13
I want to add a background job	14
7. Run the project	15
First-time setup	15
Start the core stack	15
Verify the app	15
Start observability	16
See logs	16
Stop or restart	16
8. Migrations, data, and database reset	17
Check and apply migrations	17

Inspect the local database safely	17
Reset only the local PostgreSQL database	17
9. Test and verification map	18
Fast checks by subsystem	18
Behavior tests	18
AI/BullMQ resilience smoke	18
Before saying “done”	18
10. Debugging without guessing	20
Start with the layer that is failing	20
Useful commands	20
Common messages in the checked stack	20
Do not “fix” these by weakening boundaries	20
11. Working safely with the team	21
A good change handoff contains	21
Git habits	21
Secrets	21
12. Beginner onboarding path	21
First hour	21
First day	22
First small task	22
13. Glossary	22
14. Where to go next	23

How the Project Works

A beginner-friendly field guide to the Nexora LMS/LXP

This guide assumes you are new to the repository. You do not need to understand every framework before you begin. Start with the big picture, run the stack once, then use the “where do I edit?” recipes as you take on a task.

1. Executive summary

Nexora is a learning management system (LMS) and learning experience platform (LXP) for Gat Andres Bonifacio High School. It gives administrators, teachers, and students different workspaces while keeping one backend as the source of truth.

In ordinary language:

- administrators manage people, roles, sections, classes, templates, events, diagnostics, and reports
- teachers manage classes, lessons, assessments, class records, intervention work, and reviewable AI drafts
- students read lessons, answer assessments, view performance, follow LXP/remedial work, and use the grounded “Ja” AI learning tools
- the backend decides who may do what and owns official school records
- the AI service helps generate or explain content, but it does not become the authority for grades, enrollment, or roles
- background jobs use Redis and BullMQ so long work can survive a request ending or a process restarting

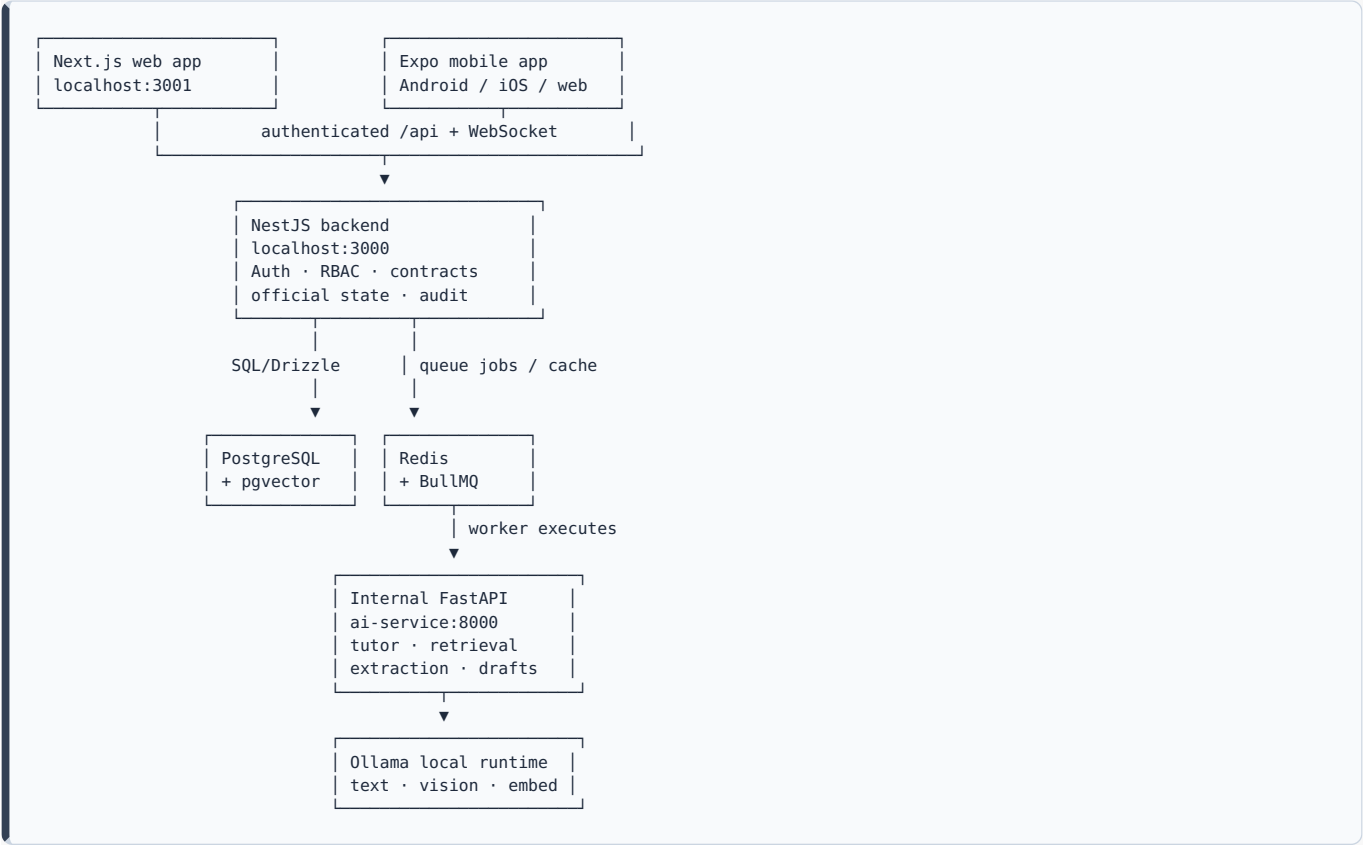
The checked local stack has six core services and eight optional observability services. On July 18, 2026, every declared healthcheck passed, the frontend returned HTTP 200, all 10 Prometheus targets were up, PostgreSQL had 89 public tables with pgvector 0.8.4, four migrations were applied, NestJS mapped 385 routes, and FastAPI exposed 57 internal paths.

The one rule to remember

Web and mobile talk to NestJS. NestJS owns public trust and official state. FastAPI is internal and assistive.

2. How the pieces fit together

The request path



What each technology contributes

Technology	What it does here	Beginner translation
NestJS 11	Public API, guards, services, WebSockets, workers	The central rule keeper
Next.js 16 + React 19	Browser application	The web screens groupmates see
Expo 54 + React Native 0.81	Mobile application	The Android/iOS screens
FastAPI	Internal AI execution	The Python service that talks to models
PostgreSQL 16	Durable relational data	The official database
pgvector	Embedding-vector storage/query support	Helps find relevant learning content
Drizzle ORM	Typed schema and SQL access	How TypeScript describes and queries tables
Redis 7	Queue transport and short-lived coordination	Fast operational memory
BullMQ	Durable job producers/workers	The background-work manager
Ollama	Local text, vision, and embedding models	The local AI model server
Prometheus/Grafana	Metrics and dashboards	“How is the system behaving?”
Loki/Promtail	Centralized logs	“What did services say?”
Tempo/OpenTelemetry	Traces	“Where did one request spend time?”

Trust boundaries

Boundary	Allowed	Not allowed
Browser/mobile → backend	Public <code>/api</code> routes with JWT/cookie/mobile tokens	Direct database or FastAPI access
Backend → PostgreSQL	Drizzle through <code>DatabaseService</code>	Client-side SQL
Backend → FastAPI	Internal shared secret plus forwarded user context	Treating the shared secret as public user auth
FastAPI → academic records	Read allowed context and write permitted AI workflow state	Directly changing official grades, roles, or enrollment
BullMQ worker → domain	Retry-safe, bounded, idempotent work	Untracked detached long-running tasks

3. The repository at a glance

capstone-nest-backend/	
├─ backend/	NestJS API, Drizzle, workers, migrations
├─ next-frontend/	Next.js web app
├─ mobile/	Expo mobile app
├─ ai-service/	FastAPI AI and retrieval service
├─ monitoring/	Prometheus, Grafana, Loki, Tempo config
├─ load-tests/	k6 and AI-pipeline smoke tests
├─ docs/	Guides, manuals, audits, plans
├─ .agents/	Repo-specific Codex workflows
├─ openspec/	Design/change artifacts
├─ docker-compose.yml	Core + optional observability services
├─ docker-compose.debug.yml	Optional host ports for DB/Redis/FastAPI
├─ .env.compose.example	Template copied to root .env
├─ README.md	Main launchpad
└─ CURRENT_REPO_STATE.md	Latest checked state

Start at these files

If you need...	Open...
Whole-project startup	<code>README.md</code>
Latest checked facts	<code>CURRENT_REPO_STATE.md</code>
Exact July 18 routes/ports	<code>docs/system-audit/2026-07-18-live-stack-and-route-inventory.md</code>
Backend rules	<code>backend/AGENTS.md</code> and <code>backend/README.md</code>
Web rules	<code>next-frontend/AGENTS.md</code> and <code>next-frontend/README.md</code>
Mobile rules	<code>mobile/AGENTS.md</code> and <code>mobile/README.md</code>
AI rules	<code>ai-service/AGENTS.md</code> and <code>ai-service/README.md</code>
Exhaustive engineering reference	<code>docs/master-manual/</code>

Dated audits and plans explain what was true when they were written. They are useful evidence, but code, migrations, AGENTS files, and current READMEs win when facts differ.

4. What happens when a user signs in

Web session

1. The browser sends credentials to `POST /api/auth/login`.

2. The backend validates the account and roles.
3. The access token stays in browser memory.
4. The refresh token is held in an HTTP-only cookie, so normal JavaScript cannot read it.
5. `AuthProvider` fetches `/api/auth/me` and chooses the correct role workspace.
6. If an API call gets 401, the shared client joins one refresh operation and retries once.
7. Backend guards still make the final authorization decision; hiding a menu is not security.

Mobile session

1. The app uses `/api/auth/mobile/login`.
2. Access and refresh tokens are returned as JSON.
3. Tokens are stored using SecureStore, with the repository's compatibility storage behavior.
4. The app hydrates the session, rotates through `/api/auth/mobile/refresh`, then fetches `/api/auth/me`.
5. Role resolution chooses admin first, teacher second, then student fallback.
6. Backend JWT, role, ownership, and enrollment checks remain authoritative.

Why refresh rotation exists

Refresh tokens are rotated so a used token cannot remain indefinitely reusable. A short 45-second grace path handles benign simultaneous refreshes. A reuse outside that boundary is treated as suspicious and can revoke the session family.

5. Understanding the core features

5.1 Classes, modules, and lessons

A class connects a teacher, section, schedule, enrolled students, learning content, assessments, and records. Modules organize material. Module sections and items can point to lessons or assessments. Lessons hold structured blocks, versions, and completion state.

Typical teacher flow:

1. create or open a class
2. add modules/sections
3. create lessons and assessments
4. publish material
5. monitor completion and results
6. use the class record and performance tools

The backend owns these relationships. Web/mobile present them and invalidate cached queries after changes.

5.2 Class Record

The Class Record is the official grade workbook. Its five core tables are:

Table	Meaning
<code>class_records</code>	One workbook per class and grading period
<code>class_record_categories</code>	Weighted groups such as Written Work or Performance Tasks
<code>class_record_items</code>	Scored columns/activities, optionally linked to assessments
<code>class_record_scores</code>	Each student's score for each item
<code>class_record_final_grades</code>	Reviewed/finalized grade outcome and lifecycle data

A simplified calculation is:

```
category percentage = points earned in category / possible points in category
weighted contribution = category percentage × category weight
initial grade = sum of weighted contributions
final/transmuted grade = backend-approved grading rule applied to initial grade
```

Important behavior:

- category weights and item maximums are backend academic state
- assessment-linked items can synchronize reviewed outcomes into the record
- bulk score writes are preferred over one request/query per cell
- finalization/reopen actions are auditable lifecycle transitions
- AI and LXP never write official class-record scores
- the displayed computed total must not become a competing unofficial source of truth

Where to work:

- schema: `backend/src/drizzle/schema/class-record.schema.ts`
- backend: `backend/src/modules/class-record/`
- teacher web route: `next-frontend/app/(dashboard)/dashboard/teacher/class-record/`
- shared hook: `next-frontend/src/hooks/use-teacher-class-record.ts`

5.3 Grading formulas

Grading formulas combine weighted categories and transmutation rules. When changing them:

1. identify whether the rule is class-specific, grading-period-specific, or template-derived
2. preserve decimal/rounding behavior
3. test zero possible points, missing scores, excused/missing state, and partial work
4. verify finalization does not silently recalculate reviewed history
5. update exports/reports and mobile/web displays if the contract changes
6. audit any operation that changes official results

Do not duplicate the formula in three clients. Keep the authoritative calculation in backend services and return typed results.

5.4 Assessments

Assessments include questions, options, publication state, attempts, responses, review, analytics, and class-record integration.

Typical lifecycle:

Teacher drafts → publishes → student starts attempt → saves answers
→ submits → objective grading / teacher review → result released
→ performance recompute → possible LXP/intervention follow-up

Assessments may be linked into module content and class-record items. Reviewed-score immutability and reopening rules must remain explicit.

Main locations:

- schema: `backend/src/drizzle/schema/base.schema.ts` and related modules
- backend: `backend/src/modules/assessments/`
- web: role pages under `next-frontend/app/(dashboard)/dashboard/*/assessments/`
- mobile: assessment screens and `mobile/src/api/services/`

5.5 Performance recompute

Performance snapshots are derived summaries, not the only official score record. The `performance-recompute` queue recalculates them after relevant academic changes.

Why it is asynchronous:

- submissions or bulk class-record edits can affect many summaries
- the user request should not wait for every aggregation
- retrying must be safe
- one deterministic job identity can deduplicate repeated triggers

Flow:

```
academic write → durable state committed → recompute job enqueued
→ worker reads authoritative records → snapshot/log updated
→ LXP/readiness consumers see refreshed derived state
```

Never replace recomputation with a permanent “eligible” boolean that can quietly go stale.

5.6 LXP and interventions

The LXP helps learners who need additional support through lessons, checkpoints, guided assessments, mastery, and intervention cases.

Key rules:

- access/eligibility comes from backend performance logic, not one AI answer
- low performance should be evaluated across meaningful evidence
- intervention history is durable and auditable
- teacher/admin approval rules govern official activation and assignment actions
- remedial content is assistive
- LXP progress does not rewrite the Class Record

Relevant schema lives mainly in `backend/src/drizzle/schema/lxp.schema.ts`. Backend behavior is in `backend/src/modules/lxp/` and `backend/src/modules/performance/`.

5.7 AI Mentor, quizzes, and extraction

“Ja” is Nexora’s grounded AI learning assistant. The backend authorizes the user and supplies permitted context. FastAPI retrieves relevant content, assembles prompts, calls the model, validates/sanitizes output, and returns assistive data.

Active feature families include:

- student tutor sessions and answer feedback
- JA practice, review, and ask
- mentor explanations
- teacher lesson-plan and quiz drafts
- intervention recommendations
- PDF/module extraction
- class/library indexing and retrieval
- admin analytics chat

AI output is a draft or explanation until an authorized backend workflow applies it. Prompt injection filtering, source scoping, retrieval thresholds, class policy, and deterministic response validation all matter.

5.8 BullMQ asynchronous jobs

The live backend registers seven queues:

Queue	Responsibility	Worker concurrency
<code>ai-teacher-generation</code>	Lesson plan, quiz, intervention, extraction	2
<code>announcements</code>	Notification fan-out	3

Queue	Responsibility	Worker concurrency
discussion-board	Discussion notifications/events	3
library-indexing	Uploaded library content indexing	2
notifications	Assessment notification dispatch	3
performance-recompute	Derived performance refresh	3
rag-indexing	Class retrieval index rebuild	1

A robust queue flow has:

1. durable intent/state in PostgreSQL when the user needs to poll it
2. a bounded payload with IDs, not giant untrusted blobs
3. a deterministic job ID where deduplication matters
4. retry/backoff and retention rules
5. an idempotent worker
6. explicit failure/cancellation state
7. metrics/logs that let operators diagnose it

Redis job state is operational evidence, not the only business record.

6. Day-to-day developer guide: where do I edit?

I want to add or change a web page

1. Find the route under `next-frontend/app/`.
2. Keep role pages under the correct `(dashboard)/dashboard/admin|teacher|student` tree.
3. Reuse the matching page shell, components, service wrappers, and theme tokens.
4. Put backend transport in `next-frontend/src/services/` or the existing shared API layer.
5. Add/update types and validation schemas.
6. Add a focused Jest test; use Playwright for a browser flow.
7. Run:

```
npm --prefix next-frontend run lint
npm --prefix next-frontend run test -- --runInBand --detectOpenHandles
npm --prefix next-frontend run build
```

Do not call FastAPI or PostgreSQL from a page.

I want to add or change a mobile screen

1. Add/find the screen in `mobile/src/screens/`.
2. Register the route in `mobile/src/navigation/` and update route-param types.
3. Put transport in `mobile/src/api/services/`.
4. Use React Query hooks and invalidate the exact keys after mutations.
5. Confirm the selected role can actually reach the screen.
6. Run:

```
npm --prefix mobile run typecheck
npm --prefix mobile run test -- --runInBand
```

On Linux/macOS, set `EXPO_PUBLIC_API_URL` and use `npm run start` or `npm run android`. The committed `android:emulator` helpers use Windows command-shell syntax.

I want to add an API endpoint

1. Choose the owning module under `backend/src/modules/`.
2. Write the failing controller/service/DTO test first.
3. Add or update a DTO with `class-validator` rules.
4. Put business logic in the service.
5. Keep the controller focused on route, auth, validation, delegation, and envelope.
6. Add `@Roles` /ownership checks and audit sensitive writes.
7. Preserve the `success/message/data` contract unless this is an approved contract change.
8. Trace web/mobile/FastAPI consumers.
9. Run:

```
npm --prefix backend run lint
npm --prefix backend run build
npm --prefix backend run test -- --runInBand
```

Swagger is available at `/api/docs` only when the backend is not running in production mode.

I want to create a database table

- 1. Add the Drizzle table under `backend/src/drizzle/schema/`.
- 2. Export it from the schema index.
- 3. Decide keys, nullability, indexes, uniqueness, and delete behavior explicitly.
- 4. Generate a forward migration from `backend/` :

```
cd backend
npx drizzle-kit generate --name describe_the_change
npm run check:migrations
```

- 5. Read the generated SQL and `drizzle/meta/_journal.json`.
- 6. Test against a disposable database.
- 7. Apply with the repository runner:

```
cd ..
node backend/run-migrations.js
```

Never edit an already-applied migration to “fix” a shared database. Never use `drizzle-kit push` on shared or deployed data.

I want to modify an AI prompt

Prompt owners are not all in one file:

Feature	Main prompt owner
Student tutor	<code>ai-service/app/student_tutor_service.py</code>
Mentor explanation	<code>ai-service/app/mentor_service.py</code>
Lesson-plan draft	<code>ai-service/app/lesson_plan_service.py</code>
Quiz draft	<code>ai-service/app/quiz_generation_service.py</code>
Intervention recommendation	<code>ai-service/app/remedial_service.py</code>
Extraction/structure	<code>ai-service/app/extraction_pipeline.py</code>
JA/admin/demo orchestration	<code>ai-service/app/main.py</code>
Prompt-injection classification	<code>ai-service/app/content_sanitizer.py</code>

Safe prompt workflow:

- 1. write or update an evaluation/test case first
- 2. state the allowed source scope and required output schema
- 3. keep user/file text clearly separated from system instructions
- 4. sanitize and length-bound untrusted text
- 5. validate model JSON/structure deterministically

6. test missing runtime, weak retrieval, timeout, unsafe input, and malformed output

7. run:

```
cd ai-service
.venv/bin/python scripts/run_tests.py
python3 scripts/run_eval_suite.py
```

If a request/response or header changes, verify `backend/src/modules/ai-mentor/ai-proxy.service.ts` and client consumers too.

I want to add a background job

Start in the backend owning module:

- register the queue with `BullModule.registerQueue`
- define a typed payload
- write a producer and `@Processor` worker
- use deterministic IDs if duplicate work is unsafe
- persist business-visible lifecycle state
- throw retryable failures back to BullMQ
- add tests for retry, duplicate delivery, cancellation, restart, and idempotency
- add metrics/log context without secrets or student PII

For long AI work, the worker may call a protected FastAPI internal execution route. FastAPI must not start its own untracked background copy.

7. Run the project

First-time setup

From the repository root:

```
cp .env.compose.example .env
```

Open `.env` and replace every `CHANGE_ME` value. At minimum, set secure values for:

- `POSTGRES_PASSWORD`
- `BACKEND_DATABASE_URL`
- `AI_DATABASE_URL`
- `JWT_SECRET`
- `JWT_REFRESH_SECRET`
- `OTP_PEPPER`
- `AI_SERVICE_SHARED_SECRET`
- `GRAFANA_ADMIN_USER` and `GRAFANA_ADMIN_PASSWORD` if using observability

Generate secrets with a secure local tool such as:

```
openssl rand -hex 32
```

Use different values for JWT access, JWT refresh, OTP pepper, database password, and internal AI secret.

Start the core stack

```
docker compose config --quiet
docker compose up -d
docker compose ps
```

The first start is slower because Ollama pulls the configured models. Follow it with:

```
docker compose logs -f ollama
```

Press Ctrl+C to stop following logs; containers continue running.

Verify the app

```
curl --fail http://localhost:3000/api/health/live
curl --fail http://localhost:3000/api/health/ready
curl --fail http://localhost:3001
docker compose exec -T redis redis-cli ping
docker compose exec -T postgres pg_isready -U postgres -d capstone
```

Open `http://localhost:3001` in a browser.

Start observability

```
docker compose --profile observability up -d
docker compose --profile observability ps
```

Open:

- Grafana: <http://localhost:3002>
- Prometheus targets: <http://localhost:9090/targets>
- Loki readiness: <http://localhost:3100/ready>
- Tempo readiness: <http://localhost:3200/ready>

The app does not require the observability profile.

See logs

```
# Everything in the currently selected core topology
docker compose logs --tail=100

# One or more services
docker compose logs --tail=200 backend ai-service
docker compose logs -f backend

# Optional profile services
docker compose --profile observability logs --tail=100 prometheus grafana loki tempo
```

Stop or restart

```
docker compose stop
docker compose start

# Or recreate containers while keeping named data volumes
docker compose down
docker compose up -d
```


8. Migrations, data, and database reset

Check and apply migrations

From the root:

```
npm --prefix backend run check:migrations
node backend/run-migrations.js
```

The live checked database had these four applied migrations:

1. 0000_baseline_nexora.sql
2. 0001_mixed_morgan_stark.sql
3. 0002_small_photon.sql
4. 0003_enable_pgvector.sql

Compose also runs the migration runner during backend startup when `RUN_DB_MIGRATIONS=true`.

Inspect the local database safely

```
docker compose exec -T postgres psql -U postgres -d capstone -c 'dt'
docker compose exec -T postgres psql -U postgres -d capstone -c 'SELECT filename, applied_at FROM _applied_migrations ORDER BY applied_at;'
```

Read before writing. Do not run ad-hoc UPDATE/DELETE statements against shared data.

Reset only the local PostgreSQL database

Danger: the following removes the current local Compose database volume. It permanently deletes local users, classes, grades, attempts, AI workflow rows, and migration history. Do not run it against a shared machine or deployment. Back up anything important first.

For this repository's current default Compose project name:

```
docker compose down
docker volume rm capstone-nest-backend_postgres_data
docker compose up -d
docker compose logs --tail=100 backend postgres
curl --fail http://localhost:3000/api/health/ready
```

If your clone directory or `COMPOSE_PROJECT_NAME` differs, first discover the exact database volume without deleting it:

```
docker volume ls --filter label=com.docker.compose.volume=postgres_data
```

Remove only the single volume you verified. Do not use a wildcard. `docker compose down --volumes` is broader: it removes database, uploads, Ollama models, and observability data, so it is not the normal database-reset command.

To add demo data after a disposable reset:

```
cd backend
node seed-database.js
npm run seed:smoke
```

Seeding is explicit; `RUN_DB_SEED` defaults to false.

9. Test and verification map

Fast checks by subsystem

```
# Backend
npm --prefix backend run check:src-clean
npm --prefix backend run check:migrations
npm --prefix backend run lint
npm --prefix backend run build

# Web
npm --prefix next-frontend run lint
npm --prefix next-frontend run build

# Mobile
npm --prefix mobile run typecheck

# AI
(cd ai-service && .venv/bin/python scripts/run_tests.py)

# Compose
docker compose config --quiet
docker compose --profile observability config --quiet
```

Behavior tests

```
npm --prefix backend run test -- --runInBand
npm --prefix backend run test:e2e -- --runInBand
npm --prefix next-frontend run test -- --runInBand --detectOpenHandles
npm --prefix next-frontend run test:e2e
npm --prefix mobile run test -- --runInBand
```

Use the smallest set that covers the change, then broaden when contracts, auth, schema, navigation, or shared orchestration changed.

AI/BullMQ resilience smoke

```
./load-tests/run-ai-pipeline-resilience-smoke.sh
```

This targets extraction, retrieval/indexing, degraded behavior, shared-secret boundaries, job identity, retry/cancellation, and tutor/JA fallbacks without intentionally loading a production model environment.

Before saying “done”

- the focused failing test now passes
- the relevant build/typecheck passes
- migration integrity passes when schema is involved
- Compose config parses when environment/topology is involved

- client and server contracts agree
- no unrelated dirty files were overwritten
- logs contain no new fatal or silent retry loop
- docs and examples name commands that actually exist

10. Debugging without guessing

Start with the layer that is failing

Page looks wrong?

→ browser console/network → frontend component/service → backend response

API returns 401/403?

→ token/cookie → refresh flow → guard/role → ownership/enrollment

Backend is not ready?

→ /api/health/ready → backend logs → named dependency logs

AI job is stuck?

→ backend job state/log → Redis queue keys → FastAPI readiness/log → Ollama

Data looks stale?

→ mutation succeeded? → query invalidated? → recompute job? → authoritative DB rows?

Useful commands

```
docker compose ps --all
docker compose logs --tail=200 backend
docker compose logs --tail=200 ai-service ollama
docker compose logs --tail=200 postgres redis
docker compose exec -T redis redis-cli --scan --pattern 'bull:*:meta'
curl -i http://localhost:3000/api/health/ready
```

Common messages in the checked stack

Message	Current meaning
Optional AWS/OTP email variables missing	Local storage/email-disabled mode can still be valid
Redis <code>vm.overcommit_memory</code> warning	Host tuning recommendation; requires operator authority
Ollama context clamped from 8192 to 2048	Loaded model reports a smaller trained context
cAdvisor cannot find Podman/CRI-O sockets	Informational on this Docker-only host
Tempo single-binary/exposure warnings	Expected local deployment caution
Loki “entry too far behind”	An old replayed log fell outside ingestion window
<code>/api/docs</code> is 404 in Compose	Expected because backend is in production mode

A warning is not automatically harmless forever. Compare it with the current-state document and investigate anything new, repeated, or tied to a failing probe.

Do not “fix” these by weakening boundaries

- do not publish FastAPI so the web can call it
- do not skip backend guards because the UI hides a button
- do not disable readiness because one dependency is misconfigured
- do not delete Redis BullMQ keys before identifying the job
- do not rewrite applied migration history
- do not store access tokens in browser localStorage

- do not write AI output directly into official grades

11. Working safely with the team

A good change handoff contains

- what changed and why
- exact files/modules owned
- public/schema/queue contract impact
- commands actually run and their result
- migrations or environment changes
- known limitations and rollback path
- screenshots only when a visual change needs evidence

Git habits

```
git status --short
git branch --show-current
git diff --check
git diff --stat
```

Preserve another groupmate's uncommitted work. Do not use destructive reset/checkout commands unless the owner explicitly asks.

Secrets

Never commit:

- root or subsystem `.env` files
- JWT, refresh, OTP, database, SMTP, cloud, or AI shared secrets
- real student credentials or PII
- copied production logs containing sensitive data

When checking environment alignment, print key names or "set/unset," not values.

12. Beginner onboarding path

First hour

- read sections 1–3 of this guide
- create root `.env` from the template
- start the core stack
- make all three public checks pass: backend live, backend ready, frontend 200
- open the web app
- inspect `docker compose ps` and one service log

First day

- trace one page from route → service wrapper → Nest controller → Nest service → Drizzle table
- trace one assessment or lesson flow
- read the relevant subsystem `AGENTS.md`
- run one focused test
- open Grafana/Prometheus if working on runtime behavior

First small task

Choose one low-risk, well-tested slice:

- improve copy/layout without changing contracts
- add a focused component test
- fix a documented empty/error state
- add a missing runbook clarification
- add a backend unit test around an existing rule

Avoid making your first task a migration, auth rewrite, grading formula change, or queue redesign without pairing/review.

13. Glossary

Term	Meaning
LMS	Core learning-management functions: classes, lessons, assessments, records
LXP	Personalized learning/remedial experience layered on official LMS data
RBAC	Role-based access control
DTO	Validated request/response shape in NestJS
Drizzle	TypeScript ORM/schema toolkit used by the backend
Migration	Ordered SQL that moves a database forward
pgvector	PostgreSQL extension for embedding vectors
RAG	Retrieval-augmented generation: fetch relevant sources before generation
BullMQ	Redis-backed queue library
Worker	Process code that consumes background jobs
Idempotent	Safe to retry without creating an incorrect duplicate effect
JWT	Signed access token used by protected API routes
Refresh rotation	Replacing refresh tokens after use to reduce replay risk
FastAPI	Internal Python AI service framework
Ollama	Local model runtime
Readiness	“Can this service safely receive traffic with dependencies working?”
Liveness	“Is the process running?”
Official state	Backend-owned school record requiring auth/audit discipline

Term	Meaning
Derived state	Recomputable summary such as a performance snapshot
Assistive output	AI/LXP content that does not become official merely because it exists

14. Where to go next

- main launchpad: `README.md`
- current facts: `CURRENT_REPO_STATE.md`
- backend setup: `backend/BACKEND_SETUP.md`
- AI workflow: `backend/AI_MENTOR_README.md`
- exact live routes: `docs/system-audit/2026-07-18-live-stack-and-route-inventory.md`
- exhaustive engineering manual: `docs/master-manual/nexora-master-service-manual.pdf`
- observability: `monitoring/README.md`

If this guide and the code disagree, treat that as documentation drift: verify the live behavior, then update the guide and the relevant source-of-truth document together.